Lab 00 — Hands-on lab: a tour of Linux from the terminal

Goal: get your hands on every idea from the theory — processes, signals, exit codes, permissions, environment, streams, ports, archives — using nothing beyond what Ubuntu 24.04 ships with. There are no containers here, but everything you do will come back unchanged in the Docker labs.

Prerequisites — Windows 10/11 with WSL 2 and an **Ubuntu 24.04** distribution. Nothing else: no Podman yet (that's lab 01), no extra packages. Open an Ubuntu terminal and stay there.

Step 0 — Where am I, and who am I?

```
head -n 2 /etc/os-release
uname -r
whoami
id
```

Observe: `PRETTY_NAME="Ubuntu 24.04.x LTS"`; a kernel like `6.6.87.2-microsoft-standard-WSL2` (the suffix is the WSL signature); your username; and a line reading `uid=1000(...) gid=1000(...) groups=... 27(sudo) ...`.

Explanation. You have just seen three identities you should never confuse again: the **distribution** (Ubuntu 24.04 — the userland), the **kernel** (built by Microsoft for WSL), and **you** (UID 1000, member of the `sudo` group). As far as the kernel is concerned, you are that number: 1000.

> **→ WINDOWS / WSL**
> If `uname -r` does not end in `-microsoft-standard-WSL2`, you are not inside WSL 2. Check from PowerShell with `wsl --version` and `wsl --list --verbose`. The entire lab series assumes WSL 2.

Step 1 — The kernel, the userland, and the line between them

```
cat /proc/version
type ls
type cd
which cat
```

Observe: the kernel version spelled out in full; `ls is /usr/bin/ls` — a program, a file on disk; `cd is a shell builtin` — not a program at all, but a function inside the shell; and `/usr/bin/cat`.

Explanation. Everything you type is either a **program** from the userland (an executable file somewhere on disk) or a builtin of the shell. Neither touches the hardware — both go through the kernel's system calls. And `cd` is a builtin for a good reason: the current directory is a property *of the shell process itself*. An external `cd` program would change its own directory and then exit, leaving yours untouched.

> **→ LINUX**
> `type` asks the shell "what would you do with this word?", while `which` only searches the `PATH`. When a command seems to be lying to you (an alias, a function), trust `type` — it always tells the truth.

Step 2 — One tree, many mounts

```
ls /
findmnt /
df -h /
ls /mnt/c/Windows 2>/dev/null | head -n 3
```

Observe: a single root (`bin boot dev etc home ... proc ... tmp usr var`); a `findmnt` line like `/ /dev/sdc ext4 rw,relatime,...`; a disk size around `1007G` (the *virtual* size of the WSL disk); and — this is Windows seen from Linux — the contents of `C:\Windows`.

```
findmnt -t proc
ls /proc | head -n 8
grep MemTotal /proc/meminfo
```

Observe: `/proc proc proc rw,relatime` — a filesystem of type `proc` with no disk behind it. The `ls` lists **numbers**, one per living process, and `MemTotal` comes straight from the kernel.

Explanation. No `C:` or `D:` drives here. Everything hangs off one tree, attached by **mounts**: the Linux disk provides `/`, the Windows disk is mounted at `/mnt/c`, and the kernel itself is mounted at `/proc` — a directory whose files are generated on the fly every time you read them. Docker images (lab 02) and volumes (lab 06) are, at bottom, just more mounts added to this tree.

> **→ WINDOWS / WSL**
> Every access to `/mnt/c` crosses the Windows ↔ Linux boundary, and that is **slow**. Projects you compile and images you store belong on the Linux side (`/home/...`), not in `/mnt/c/Users/...`. Make that a habit now.

Step 3 — Processes: PID, parent, /proc

```
ps
echo $$
ps -p 1 -o pid,comm
```

Observe: `ps` shows almost nothing (your `bash` and the `ps` itself); `echo $$` prints your shell's PID; and process 1 is `systemd`. On its own, `ps` only lists the processes of *your terminal*. All the others — `ps -ef` shows them — run without a terminal, and most of them are **daemons**: service processes like `systemd` itself, with names that usually end in "d".

Now start a process that sticks around, in the background:

```
sleep 300 &
ps -o pid,ppid,stat,cmd
```

Observe a `sleep 300` line whose **PPID equals your bash's PID** — a parent-child link, live:

```
  PID  PPID STAT CMD
 2363  2362 S    bash
 2419  2363 S    sleep 300
 2420  2363 R    ps -o pid,ppid,stat,cmd
```

Go look at that process in `/proc` (replace `2419` with your own PID):

```
head -n 3 /proc/2419/status
tr '\0' ' ' < /proc/2419/cmdline; echo
ls -l /proc/2419/exe
```

Observe: `Name: sleep`, `State: S (sleeping)`, the exact command line, and a link `exe -> /usr/bin/sleep`.

Explanation. There is no magic in `ps` — it just reads `/proc`. Everything Docker will show you later (`podman top`, `podman inspect`) comes from the same place. `STAT S` means *sleeping* (waiting); `R` means *running*.

Step 4 — Signals and exit codes

The `sleep` is still running. Ask it nicely to leave:

```
kill 2419        # your own PID here
ps -o pid,cmd | grep "[s]leep 300" || echo "no more sleep process"
```

Observe: the shell reports `Terminated`, then `no more sleep process`. Plain `kill` sends `SIGTERM`, and `sleep` complies.

Once more, the hard way:

```
sleep 300 &
kill -9 %1
```

Observe: this time the shell reports `Killed`. `SIGKILL` didn't ask. (`%1` refers to the shell's *job* number 1 — handy when you don't feel like hunting down the PID.)

Now collect the exit codes:

```
true;  echo $?
false; echo $?
ls /does-not-exist; echo $?
unknown-command; echo $?
bash -c 'kill -9 $$'; echo $?
```

Observe, in order: `0`, `1`, `2` (after the `ls` error message), `127` (after `command not found`), and **`137`** (after `Killed`).

Explanation. `0` means success, everything else means failure, and `128 + n` means death by signal *n* — so 137 = 128 + 9 = killed by SIGKILL. These five numbers are exactly what `podman ps` will show in its `Exited (...)` column in lab 03. Learn to read them here, where everything is simple.

REMEMBER
Escalate in the right order: `kill` first (SIGTERM — the application gets to clean up), wait, and only then `kill -9` (SIGKILL — the kernel erases it). `docker stop` runs this protocol for you: SIGTERM, a 10-second grace period, then SIGKILL.

Step 5 — The environment and the PATH

```
echo $HOME
env | wc -l
env | grep -E '^(HOME|PATH|LANG)='
```

Observe your environment: a few dozen variables, among them `HOME=/home/<you>` and a `PATH` which, on WSL, even includes Windows paths (`/mnt/c/Windows/system32`...).

Now the decisive experiment — a shell variable is **not** an environment variable:

```
MSG=hello
echo $MSG
bash -c 'echo child sees: [$MSG]'
export MSG
bash -c 'echo child sees: [$MSG]'
```

Observe: `hello`, then `child sees: []` — empty! — then, after `export`, `child sees: [hello]`.

Explanation. Every child process gets a **copy** of its parent's environment, frozen at launch. Before the `export`, `MSG` existed only inside your shell. This is the exact mechanism `podman run -e MSG=hello` will use in lab 08 to configure your applications.

Next, the `PATH`:

```
mkdir -p ~/lab0/tools
printf '#!/bin/bash\necho "homemade tool: ok"\n' > ~/lab0/tools/mytool
chmod +x ~/lab0/tools/mytool
mytool; echo $?
export PATH="$HOME/lab0/tools:$PATH"
mytool
```

Observe: first `command not found` and `127`; then, once the directory is on the `PATH`, `homemade tool: ok`.

Explanation. The shell doesn't "know" any commands. It searches the `PATH` directories in order for an executable with that name and stops at the first match. (This modified `PATH` lasts only for this shell — to make it permanent, add one line to `~/.bashrc`.)

Step 6 — Three streams: redirections and pipes

```
cd ~/lab0
echo "first line"  > notes.txt
echo "second line" >> notes.txt
cat notes.txt
```

Observe: `>` creates (or overwrites!), `>>` appends.

Now split the two output streams:

```
ls notes.txt /does-not-exist > output.txt 2> errors.txt
cat output.txt
cat errors.txt
```

Observe: the screen stayed silent during the `ls`. `output.txt` holds `notes.txt`; `errors.txt` holds `ls: cannot access '/does-not-exist': No such file or directory`.

```
ls notes.txt /does-not-exist > all.txt 2>&1
cat all.txt
```

Observe both lines together: `2>&1` plugs the error stream (2) into wherever the output stream (1) currently points.

Finally, pipes:

```
ps -ef | wc -l
ps -ef | grep "[b]ash" | head -n 3
```

Observe the system's process count, then your shells — with no temporary file anywhere: each command's output feeds the next command's input.

> **→ LINUX / SHELL**
> About that `grep "[b]ash"` trick: the brackets form a regular expression that matches `bash`, but the `grep` command line itself contains `[b]ash`, which doesn't match the pattern. Without the trick, `grep` would always find itself in the list. You will see this idiom in every lab.

Step 7 — Permissions: how to read an `ls -l`

```
ls -l notes.txt
stat -c "%U %G %a %n" notes.txt
```

Observe: `-rw-r--r-- 1 <you> <you> 23 ... notes.txt`, and its numeric form `644` — owner `rw` (6), group `r` (4), others `r` (4).

Create a script and try to run it:

```
printf '#!/bin/bash\necho "Hello, I am process $$"\n' > hello.sh
./hello.sh; echo $?
chmod +x hello.sh
ls -l hello.sh
./hello.sh
```

Observe: `Permission denied` and code **126** — found, but not executable. After `chmod +x`: `-rwxr-xr-x`, and the script runs, with a different PID every time.

Now the root boundary:

```
cat /etc/shadow; echo $?
ls -l /etc/shadow
sudo head -n 1 /etc/shadow
```

Observe: `Permission denied` (code 1); the line `-rw-r----- 1 root shadow ...` that explains it (you are neither `root` nor in the `shadow` group); and then, through `sudo`, the first line `root:*:...` (`*` or `!` means the account is locked — no password will ever match).

Explanation. The kernel compares the process's UID against the three `rwx` triplets and applies the first one that matches you. `sudo` doesn't sneak around anything: it starts the process with UID 0, and the kernel refuses UID 0 nothing. Keep this model in mind for lab 06, when a container writes files into a volume under an unexpected UID.

Step 8 — A server, a port, a client

Ubuntu 24.04 ships with Python, so your first HTTP server is one line away.

```
echo "<h1>Hello from my server</h1>" > index.html
python3 -m http.server 8080 &
curl -s http://localhost:8080/index.html
```

Observe your HTML coming back over HTTP: `<h1>Hello from my server</h1>`.

```
curl -si http://localhost:8080/index.html | head -n 4
ss -tlnp | grep 8080
```

Observe the full HTTP response (`HTTP/1.0 200 OK`, `Server: SimpleHTTP/0.6 Python/3.12.3`, `Content-type: text/html`) and the listening socket:

```
LISTEN 0  5  0.0.0.0:8080  0.0.0.0:*  users:(("python3",pid=2788,fd=3))
```

`0.0.0.0:8080`: the `python3` process is listening on **all** interfaces, port 8080.

> **→ WINDOWS / WSL**
> Open a **Windows** browser at `http://localhost:8080` — the page loads. WSL 2 forwards `localhost` from Windows to Ubuntu automatically. In lab 07, this same forwarding is what lets you test your containers from a Windows browser.

Now try a privileged port:

```
python3 -m http.server 80
```

Observe the failure: `PermissionError: [Errno 13] Permission denied`. Port 80 sits below the 1024 threshold, which belongs to root — and you are UID 1000. Rootless Podman lives with the same restriction.

Shut the server down and confirm:

```
kill %1
curl -s --max-time 2 http://localhost:8080/; echo $?
```

Observe `curl` exit code **7** — *connection refused*. Nobody is listening anymore.

Step 9 — Archives: `tar`, the ancestor of images

```
mkdir -p my-app/config
echo "app.port=8080" > my-app/config/app.properties
echo "fake binary" > my-app/app.bin
tar -czf my-app.tar.gz my-app
ls -lh my-app.tar.gz
file my-app.tar.gz
```

Observe an archive of a few hundred bytes, identified as `gzip compressed data`.

```
tar -tf my-app.tar.gz
mkdir -p /tmp/restore
tar -xzf my-app.tar.gz -C /tmp/restore
cat /tmp/restore/my-app/config/app.properties
```

Observe the content listing (`-t` for *list*), then the extraction to another location (`-C`), and the file restored bit for bit: `app.port=8080`.

Explanation. `tar` (*tape archive*, 1979) packs an entire directory tree — paths, permissions, owners — into a single file. Remember it well: a Docker image **layer** is literally a tar archive, and `podman save` (lab 02) will hand you a tar of tars. Nothing new under the sun.

Cleanup

Make sure no lab process is left behind, then delete the files:

```
ps -o pid,cmd | grep -E "[s]leep|[h]ttp.server" || echo "nothing to kill"
rm -r ~/lab0
rm -r /tmp/restore
```

The modified `PATH` and the `MSG` variable die with this shell, so just close the terminal. Nothing was installed, so there is nothing to uninstall.

What you should be able to say now

- My kernel is `...-microsoft-standard-WSL2`; my distribution is Ubuntu 24.04; I am UID 1000.
- A process has a PID and a parent. I watched one get born (`&`), live (`/proc/<pid>/`), and die (`kill`).
- `kill` sends SIGTERM, which can be negotiated; `kill -9` sends SIGKILL, which cannot. A process killed by SIGKILL exits with `137` = 128 + 9.
- `$?` is `0` on success; `126` means not executable; `127` means not found in the `PATH`.
- A variable reaches child processes only after `export` — and never reaches a process that is already running.
- `>` captures stdout, `2>` captures stderr, `2>&1` merges them, `|` chains processes together.
- `-rw-r-----` reads as three triplets; the kernel compares UIDs; `root` (UID 0) skips the checks entirely.
- `ss -tlnp` shows who listens on which port; `0.0.0.0` means all interfaces; below 1024 is root-only; `curl` tests it all.
- A mount attaches a filesystem to the single tree; `/proc` has no disk behind it; `tar` packs up a tree — which is exactly what Docker images do.